Assignment 1 - Code Submission

COSC2637 Big Data Processing

Postgraduate individual submission

Student ID	s4271512
Assessment	Assignment 1
Contents	Python source and Bash run scripts from Tasks 1, 2, and 3

Task	File	Lines
Task 1	task1_mapper.py	64
Task 1	task1_reducer.py	64
Task 1	task1-run.sh	23
Task 2	task2_mapper.py	86
Task 2	task2_reducer.py	90
Task 2	task2_driver.py	227
Task 2	task2-run.sh	11
Task 3	task3_job1_mapper.py	34
Task 3	task3_job1_reducer.py	44
Task 3	task3_job2_mapper.py	62
Task 3	task3_job2_reducer.py	94
Task 3	task3_job3_mapper.py	29
Task 3	task3_job3_reducer.py	50
Task 3	task3-run.sh	74

task1_mapper.py

Task 1 | Task 1/task1_mapper.py | 64 lines

```
#!/usr/bin/env python3
import sys
from decimal import Decimal, InvalidOperation, ROUND_HALF_UP

# Task 1 - Taxi-Level Trip Efficiency Analysis
# Input line (Trips.txt): trip_id,taxi_id,fare,distance,pickup_x,pickup_y,dropoff_x,dropoff_y
# Trip type by distance: long >= 200, medium 100-200, short < 100
#
# In-mapper combining: state (a dict keyed by taxi_id+trip_type) is preserved
# across all input lines seen by this mapper task, and partial aggregates are
# only emitted once per key after the whole input split has been read. This
# cuts the number of <key,value> pairs shipped to the reducers compared to
# emitting one pair per trip.
#
# Emitted key/value (tab separated):
#   taxi_id  trip_type  partial_count  partial_max_cents  partial_min_cents  partial_sum_cents

stats = {}  # (taxi_id, trip_type) -> [count, max_cents, min_cents, sum_cents]

for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    fields = line.split(",")
    if len(fields) != 8:
        continue  # skip malformed lines

    taxi_id, fare_text, distance_text = fields[1], fields[2], fields[3]
    try:
        # Store money as integer cents so mapper split boundaries and floating-
        # point addition cannot change the final two-decimal-place result.
        fare_cents = int(
            (Decimal(fare_text) * 100).quantize(
                Decimal("1"), rounding=ROUND_HALF_UP
            )
        )
        distance = float(distance_text)
    except (InvalidOperation, ValueError):
        continue

    if distance >= 200:
        trip_type = "long"
    elif distance >= 100:
        trip_type = "medium"
    else:
        trip_type = "short"

    key = (taxi_id, trip_type)
    if key in stats:
        s = stats[key]
        s[0] += 1
        if fare_cents > s[1]:
            s[1] = fare_cents
        if fare_cents < s[2]:
            s[2] = fare_cents
```

```
56          s[3] += fare_cents
57      else:
58          stats[key] = [1, fare_cents, fare_cents, fare_cents]
59
60  for (taxi_id, trip_type), (count, max_cents, min_cents, sum_cents) in stats.items():
61      print(
62          "%s\t%s\t%d\t%d\t%d\t%d"
63          % (taxi_id, trip_type, count, max_cents, min_cents, sum_cents)
64      )
```

task1_reducer.py

Task 1 | Task 1/task1_reducer.py | 64 lines

```
#!/usr/bin/env python3
import sys
from decimal import Decimal, ROUND_HALF_UP

# Task 1 - Reducer
# Receives partial aggregates from each mapper's in-mapper combining step,
# grouped/sorted by the compound key (taxi_id, trip_type) thanks to
# stream.num.map.output.key.fields=2 + KeyFieldBasedPartitioner in task1-run.sh.
# Final aggregation only: merge partial counts/sums/max/min and compute the average.

current_taxi = None
current_type = None
count = 0
max_cents = None
min_cents = None
sum_cents = 0


def cents_to_text(value):
    return format(Decimal(value) / Decimal(100), ".2f")


def emit(taxi_id, trip_type, count, max_cents, min_cents, sum_cents):
    avg_fare = (Decimal(sum_cents) / Decimal(100 * count)).quantize(
        Decimal("0.01"), rounding=ROUND_HALF_UP
    )
    print(
        "%s\t%s\t%d\t%s\t%s\t%s"
        % (
            taxi_id,
            trip_type,
            count,
            cents_to_text(max_cents),
            cents_to_text(min_cents),
            format(avg_fare, ".2f"),
        )
    )


for line in sys.stdin:
    line = line.strip()
    if not line:
        continue
    taxi_id, trip_type, c, mx, mn, sm = line.split("\t")
    c = int(c)
    mx = int(mx)
    mn = int(mn)
    sm = int(sm)

    if taxi_id == current_taxi and trip_type == current_type:
        count += c
        sum_cents += sm
        if mx > max_cents:
            max_cents = mx
        if mn < min_cents:
```

```
            min_cents = mn
    else:
        if current_taxi is not None:
            emit(current_taxi, current_type, count, max_cents, min_cents, sum_cents)
        current_taxi, current_type = taxi_id, trip_type
        count, max_cents, min_cents, sum_cents = c, mx, mn, sm

if current_taxi is not None:
    emit(current_taxi, current_type, count, max_cents, min_cents, sum_cents)
```

task1-run.sh

Task 1 | Task 1/task1-run.sh | 23 lines

```
#!/bin/bash
# Task 1 - Taxi-Level Trip Efficiency Analysis
# Runs end-to-end with no manual intervention: clears any previous output
# directory, then runs the streaming MapReduce job with 3 reducers.
# Compound key (taxi_id, trip_type) is grouped/partitioned as a 2-field key,
# matching the pattern taught for multi-field grouping (KeyFieldBasedPartitioner).

set -euo pipefail
cd "$(dirname "$0")"
chmod +x task1_mapper.py task1_reducer.py

hadoop fs -rm -r -f /Output/task1

hadoop jar /usr/lib/hadoop/hadoop-streaming.jar \
  -D stream.num.map.output.key.fields=2 \
  -D mapred.reduce.tasks=3 \
  -file ./task1_mapper.py \
  -mapper ./task1_mapper.py \
  -file ./task1_reducer.py \
  -reducer ./task1_reducer.py \
  -input /Input/Trips.txt \
  -output /Output/task1 \
  -partitioner org.apache.hadoop.mapred.lib.KeyFieldBasedPartitioner
```

task2_mapper.py

Task 2 | Task 2/task2_mapper.py | 86 lines

```
#!/usr/bin/env python3
"""Assign trip drop-off points to their nearest current medoid.

Input (Trips.txt):
    trip_id,taxi_id,fare,distance,pickup_x,pickup_y,dropoff_x,dropoff_y

Mapper output (tab separated):
    cluster_index    dropoff_x    dropoff_y    partial_count

The dictionary is an in-mapper combiner. Repeated coordinates assigned to the
same cluster are emitted once per mapper flush, reducing shuffle traffic while
preserving the exact multiplicity needed by the PAM update.
"""

import sys


MEDOIDS_FILE = "task2_medoids.txt"
MAX_BUFFERED_KEYS = 50000


def load_medoids(path):
    medoids = []
    with open(path, encoding="utf-8") as source:
        for line_number, line in enumerate(source, start=1):
            line = line.strip()
            if not line:
                continue
            try:
                x_text, y_text = line.split()
                medoids.append((float(x_text), float(y_text)))
            except ValueError:
                raise ValueError(
                    "Invalid medoid on line %d of %s" % (line_number, path)
                )
    if not medoids:
        raise ValueError("No medoids were supplied in %s" % path)
    return medoids


def nearest_medoid(x, y, medoids):
    """Return the lowest-index medoid at minimum Euclidean distance.

    Squared distance is sufficient for comparison and avoids an unnecessary
    square root for every medoid examined by the mapper.
    """
    best_index = 0
    best_distance_squared = None
    for index, (medoid_x, medoid_y) in enumerate(medoids):
        distance_squared = (x - medoid_x) ** 2 + (y - medoid_y) ** 2
        if best_distance_squared is None or distance_squared < best_distance_squared:
            best_index = index
            best_distance_squared = distance_squared
    return best_index

```

```
56
57  def emit_counts(point_counts):
58      for (cluster_index, x, y), count in point_counts.items():
59          print("%d\t%.3f\t%.3f\t%d" % (cluster_index, x, y, count))
60
61
62  medoids = load_medoids(MEDOIDS_FILE)
63  point_counts = {}
64
65  for line in sys.stdin:
66      fields = line.strip().split(",")
67      if len(fields) != 8:
68          continue
69
70      try:
71          dropoff_x = float(fields[6])
72          dropoff_y = float(fields[7])
73      except ValueError:
74          continue
75
76      cluster_index = nearest_medoid(dropoff_x, dropoff_y, medoids)
77      key = (cluster_index, dropoff_x, dropoff_y)
78      point_counts[key] = point_counts.get(key, 0) + 1
79
80      # Bound memory for data with many distinct coordinates. Multiple partial
81      # records for a point are safely merged by the reducer.
82      if len(point_counts) >= MAX_BUFFERED_KEYS:
83          emit_counts(point_counts)
84          point_counts.clear()
85
86  emit_counts(point_counts)
```

task2_reducer.py

Task 2 | Task 2/task2_reducer.py | 90 lines

```
#!/usr/bin/env python3
"""Perform the exact PAM medoid-update step for each assigned cluster.

Reducer input (grouped by cluster_index):
    cluster_index    dropoff_x    dropoff_y    partial_count

Internal reducer output consumed by task2_driver.py:
    cluster_index    medoid_x    medoid_y    #points    avg_dissimilarity    membership_signature

The reducer merges mapper partial counts, then tests every distinct assigned
point as the swap candidate. Counts are used as weights, so duplicate points
have exactly the same effect as processing every trip individually.
"""

import hashlib
import math
import sys


def average_dissimilarity(candidate, weighted_points, total_count):
    candidate_x, candidate_y = candidate
    total_distance = 0.0
    for (point_x, point_y), count in weighted_points.items():
        total_distance += math.hypot(candidate_x - point_x, candidate_y - point_y) * count
    return total_distance / total_count


def membership_signature(weighted_points):
    """Create a deterministic signature for assignment-convergence testing."""
    digest = hashlib.sha256()
    for (x, y), count in sorted(weighted_points.items()):
        digest.update(("%.3f,%.3f:%d;" % (x, y, count)).encode("ascii"))
    return digest.hexdigest()


def choose_medoid(weighted_points):
    """Return the assigned data point with minimum weighted average distance."""
    total_count = sum(weighted_points.values())
    candidates = sorted(weighted_points)
    best_candidate = candidates[0]
    best_cost = average_dissimilarity(best_candidate, weighted_points, total_count)

    for candidate in candidates[1:]:
        cost = average_dissimilarity(candidate, weighted_points, total_count)
        if cost < best_cost:
            best_candidate = candidate
            best_cost = cost

    return best_candidate, total_count, best_cost


def emit_cluster(cluster_index, weighted_points):
    if cluster_index is None or not weighted_points:
        return
    medoid, total_count, cost = choose_medoid(weighted_points)
```

```
    signature = membership_signature(weighted_points)
    print(
        "%d\t%.3f\t%.3f\t%d\t%.2f\t%s"
        % (cluster_index, medoid[0], medoid[1], total_count, cost, signature)
    )


current_cluster = None
weighted_points = {}

for line in sys.stdin:
    fields = line.strip().split("\t")
    if len(fields) != 4:
        continue

    try:
        cluster_index = int(fields[0])
        point = (float(fields[1]), float(fields[2]))
        partial_count = int(fields[3])
    except ValueError:
        continue

    if partial_count <= 0:
        continue

    if current_cluster is None:
        current_cluster = cluster_index
    elif cluster_index != current_cluster:
        emit_cluster(current_cluster, weighted_points)
        current_cluster = cluster_index
        weighted_points = {}

    weighted_points[point] = weighted_points.get(point, 0) + partial_count

emit_cluster(current_cluster, weighted_points)
```

task2_driver.py

Task 2 | Task 2/task2_driver.py | 227 lines

```
#!/usr/bin/env python3
"""Orchestrate iterative three-reducer Hadoop Streaming jobs for Task 2.

Hadoop MapReduce is not itself iterative, so this driver performs the control
flow: read v and the k initial medoids, run one assignment/update job per PAM
iteration, read the k small reducer summaries, print the required progress,
test whether assignments changed, and publish the final k lines to HDFS.
"""

import os
import subprocess
import sys


HADOOP_STREAMING_JAR = "/usr/lib/hadoop/hadoop-streaming.jar"
INPUT_PATH = "/Input/Trips.txt"
FINAL_OUTPUT = "/Output/task2"
TEMP_OUTPUT_BASE = "/tmp/bdp_assignment1_task2"
INITIALIZATION_FILE = "initialization.txt"
MEDOIDS_FILE = "task2_medoids.txt"
LOCAL_FINAL_FILE = "task2_final_output.txt"
NUM_REDUCERS = 3


def run(command, capture_output=False):
    result = subprocess.run(
        command,
        text=True,
        capture_output=capture_output,
        check=False,
    )
    if result.returncode != 0:
        if capture_output and result.stderr:
            print(result.stderr, file=sys.stderr, end="")
        sys.exit(
            "Command failed with exit code %d: %s"
            % (result.returncode, " ".join(command))
        )
    return result


def read_initialization(path):
    with open(path, encoding="utf-8") as source:
        lines = [line.strip() for line in source if line.strip()]

    if len(lines) < 2:
        raise ValueError("initialization.txt must contain v and at least one medoid")

    try:
        max_iterations = int(lines[0])
    except ValueError:
        raise ValueError("The first line of initialization.txt must be an integer v")
    if max_iterations < 1:
        raise ValueError("v must be at least 1")

```

```
    medoids = []
    for line_number, line in enumerate(lines[1:], start=2):
        try:
            x_text, y_text = line.split()
            medoids.append((float(x_text), float(y_text)))
        except ValueError:
            raise ValueError(
                "Line %d of initialization.txt must contain medoid_x medoid_y"
                % line_number
            )

    return max_iterations, medoids


def write_medoids(medoids):
    with open(MEDOIDS_FILE, "w", encoding="utf-8", newline="\n") as destination:
        for x, y in medoids:
            destination.write("%.3f %.3f\n" % (x, y))


def run_iteration(iteration, medoids):
    write_medoids(medoids)
    output_path = "%s/iteration_%03d" % (TEMP_OUTPUT_BASE, iteration)

    run(["hadoop", "fs", "-rm", "-r", "-f", output_path])
    run(
        [
            "hadoop",
            "jar",
            HADOOP_STREAMING_JAR,
            "-D",
            "mapred.reduce.tasks=%d" % NUM_REDUCERS,
            "-file",
            "./task2_mapper.py",
            "-mapper",
            "./task2_mapper.py",
            "-file",
            "./task2_reducer.py",
            "-reducer",
            "./task2_reducer.py",
            "-file",
            "./%s" % MEDOIDS_FILE,
            "-input",
            INPUT_PATH,
            "-output",
            output_path,
        ]
    )
    return output_path


def read_iteration_output(output_path):
    result = run(
        ["hadoop", "fs", "-cat", output_path + "/part-*"],
        capture_output=True,
    )
    clusters = {}

    for line in result.stdout.splitlines():
        if not line.strip():
```

```
            continue
        fields = line.split("\t")
        if len(fields) != 6:
            raise ValueError("Unexpected reducer output: %s" % line)
        cluster_index = int(fields[0])
        clusters[cluster_index] = (
            float(fields[1]),
            float(fields[2]),
            int(fields[3]),
            float(fields[4]),
            fields[5],
        )

    return clusters


def publish_final_output(clusters, medoids):
    with open(LOCAL_FINAL_FILE, "w", encoding="utf-8", newline="\n") as destination:
        for index, (fallback_x, fallback_y) in enumerate(medoids):
            if index in clusters:
                medoid_x, medoid_y, count, cost, _signature = clusters[index]
            else:
                medoid_x, medoid_y, count, cost = fallback_x, fallback_y, 0, 0.0
            destination.write(
                "%.3f\t%.3f\t%d\t%.2f\n"
                % (medoid_x, medoid_y, count, cost)
            )

    run(["hadoop", "fs", "-rm", "-r", "-f", FINAL_OUTPUT])
    run(["hadoop", "fs", "-mkdir", "-p", FINAL_OUTPUT])
    run(
        [
            "hadoop",
            "fs",
            "-put",
            "-f",
            LOCAL_FINAL_FILE,
            FINAL_OUTPUT + "/part-00000",
        ]
    )


def main():
    try:
        max_iterations, medoids = read_initialization(INITIALIZATION_FILE)
    except (OSError, ValueError) as error:
        sys.exit("Initialization error: %s" % error)

    cluster_count = len(medoids)
    previous_signatures = None
    clusters = {}

    # Remove stale results before beginning so a failed run cannot be mistaken
    # for the current run's final output.
    run(["hadoop", "fs", "-rm", "-r", "-f", FINAL_OUTPUT])
    run(["hadoop", "fs", "-rm", "-r", "-f", TEMP_OUTPUT_BASE])

    for iteration in range(1, max_iterations + 1):
        output_path = run_iteration(iteration, medoids)
        clusters = read_iteration_output(output_path)
```

```
176
177         new_medoids = []
178         signatures = []
179         print("Iteration %d" % iteration)
180
181         for index in range(cluster_count):
182             if index in clusters:
183                 medoid_x, medoid_y, count, cost, signature = clusters[index]
184             else:
185                 # A cluster can be empty, especially with duplicate initial
186                 # medoids. Keep its current medoid and still emit a final line.
187                 medoid_x, medoid_y = medoids[index]
188                 count, cost, signature = 0, 0.0, "EMPTY"
189                 clusters[index] = (
190                     medoid_x,
191                     medoid_y,
192                     count,
193                     cost,
194                     signature,
195                 )
196
197             new_medoids.append((medoid_x, medoid_y))
198             signatures.append(signature)
199             print(
200                 "cluster %d\tmedoid=(%.3f, %.3f)\t#points=%d\tavg_dissimilarity=%.2f"
201                 % (index, medoid_x, medoid_y, count, cost)
202             )
203
204         assignments_unchanged = (
205             previous_signatures is not None and signatures == previous_signatures
206         )
207         medoids = new_medoids
208
209         if assignments_unchanged:
210             print("Assignments unchanged; converged after %d iteration(s)." % iteration)
211             break
212
213         previous_signatures = signatures
214
215     publish_final_output(clusters, medoids)
216     run(["hadoop", "fs", "-rm", "-r", "-f", TEMP_OUTPUT_BASE])
217     print("Final output written to %s" % FINAL_OUTPUT)
218
219     for local_temporary_file in (MEDOIDS_FILE, LOCAL_FINAL_FILE):
220         try:
221             os.remove(local_temporary_file)
222         except OSError:
223             pass
224
225
226 if __name__ == "__main__":
227     main()
```

task2-run.sh

Task 2 | Task 2/task2-run.sh | 11 lines

```
 1  #!/bin/bash
 2  # Task 2 - exact PAM/k-medoids clustering of trip drop-off locations.
 3  # This script is the required end-to-end entry point on the EMR master node.
 4
 5  set -euo pipefail
 6  cd "$(dirname "$0")"
 7
 8  # Hadoop Streaming launches these files directly on worker nodes.
 9  chmod +x task2_mapper.py task2_reducer.py
10
11  python3 ./task2_driver.py
```

task3_job1_mapper.py

Task 3 | Task 3/task3_job1_mapper.py | 34 lines

```
#!/usr/bin/env python3
"""Job 1 mapper: tag Taxis and Trips records for a reduce-side join.

Taxis mapper output: taxi_id, tag 0, company_id
Trips mapper output: taxi_id, tag 1, fare, distance

The composite key (taxi_id, tag) lets Hadoop sort taxi metadata before trip
records while partitioning only by taxi_id.
"""

import sys


for line in sys.stdin:
    fields = line.strip().split(",")

    if len(fields) == 4:  # Taxis.txt: taxi_id,company_id,model,year
        taxi_id, company_id = fields[0], fields[1]
        try:
            int(taxi_id)
            int(company_id)
        except ValueError:
            continue
        print("%s\t0\t%s" % (taxi_id, company_id))

    elif len(fields) == 8:  # Trips.txt
        taxi_id, fare_text, distance_text = fields[1], fields[2], fields[3]
        try:
            int(taxi_id)
            fare = float(fare_text)
            distance = float(distance_text)
        except ValueError:
            continue
        print("%s\t1\t%.2f\t%.2f" % (taxi_id, fare, distance))
```

task3_job1_reducer.py

Task 3 | Task 3/task3_job1_reducer.py | 44 lines

```
#!/usr/bin/env python3
"""Job 1 reducer: stream the secondary-sorted taxi/company join.

Input is partitioned by taxi_id and sorted by (taxi_id, tag), so tag 0 taxi
metadata arrives before tag 1 trips. No trip list is buffered.

Output:
    company_id, taxi_id, record_type(F or T), fare, distance

An F marker is emitted once per taxi so Job 2 can count the complete company
fleet even when a taxi has no trips.
"""

import sys


current_taxi = None
company_id = None
fleet_marker_emitted = False

for line in sys.stdin:
    fields = line.strip().split("\t")
    if len(fields) < 3:
        continue

    taxi_id, tag = fields[0], fields[1]
    if taxi_id != current_taxi:
        current_taxi = taxi_id
        company_id = None
        fleet_marker_emitted = False

    if tag == "0" and len(fields) == 3:
        company_id = fields[2]
        if not fleet_marker_emitted:
            print("%s\t%s\tF\t0.00\t0.00" % (company_id, taxi_id))
            fleet_marker_emitted = True

    elif tag == "1" and len(fields) == 4 and company_id is not None:
        try:
            fare = float(fields[2])
            distance = float(fields[3])
        except ValueError:
            continue
        print("%s\t%s\tT\t%.2f\t%.2f" % (company_id, taxi_id, fare, distance))
```

task3_job2_mapper.py

Task 3 | Task 3/task3_job2_mapper.py | 62 lines

```
#!/usr/bin/env python3
"""Job 2 mapper: create per-(company,taxi) partial aggregates.

Input from Job 1:
    company_id, taxi_id, F/T, fare, distance

Output:
    company_id, taxi_id, fleet_present, partial_revenue,
    partial_trip_count, partial_distance

State is preserved across mapper input lines and periodically flushed, so the
mapper reduces shuffle traffic without assuming the complete input fits in
memory.
"""

import sys


MAX_BUFFERED_KEYS = 50000


def emit_partials(partials):
    for (company_id, taxi_id), values in partials.items():
        fleet_present, revenue, trips, distance = values
        print(
            "%s\t%s\t%d\t%.2f\t%d\t%.2f"
            % (company_id, taxi_id, fleet_present, revenue, trips, distance)
        )


partials = {}

for line in sys.stdin:
    fields = line.strip().split("\t")
    if len(fields) != 5:
        continue

    company_id, taxi_id, record_type = fields[0], fields[1], fields[2]
    key = (company_id, taxi_id)
    if key not in partials:
        partials[key] = [0, 0.0, 0, 0.0]

    values = partials[key]
    if record_type == "F":
        values[0] = 1
    elif record_type == "T":
        try:
            fare = float(fields[3])
            distance = float(fields[4])
        except ValueError:
            continue
        values[1] += fare
        values[2] += 1
        values[3] += distance
    else:
```

```
56          continue
57
58      if len(partials) >= MAX_BUFFERED_KEYS:
59          emit_partials(partials)
60          partials.clear()
61
62  emit_partials(partials)
```

task3_job2_reducer.py

Task 3 | Task 3/task3_job2_reducer.py | 94 lines

```
#!/usr/bin/env python3
"""Job 2 reducer: aggregate taxi partials into company performance metrics.

Input is partitioned by company_id and sorted by (company_id, taxi_id), making
all partials for one taxi consecutive. The reducer streams one taxi at a time,
then emits one six-field company summary.
"""

import sys


current_company = None
current_taxi = None

taxi_present = False
taxi_revenue = 0.0
taxi_trips = 0
taxi_distance = 0.0

company_fleet = 0
company_revenue = 0.0
company_trips = 0
company_distance = 0.0


def add_current_taxi():
    global company_fleet, company_revenue, company_trips, company_distance
    if current_taxi is None:
        return
    if taxi_present:
        company_fleet += 1
    company_revenue += taxi_revenue
    company_trips += taxi_trips
    company_distance += taxi_distance


def emit_current_company():
    if current_company is None or company_fleet == 0:
        return
    revenue_per_taxi = company_revenue / company_fleet
    average_distance = company_distance / company_trips if company_trips else 0.0
    print(
        "%s\t%.2f\t%d\t%d\t%.2f\t%.2f"
        % (
            current_company,
            company_revenue,
            company_trips,
            company_fleet,
            revenue_per_taxi,
            average_distance,
        )
    )


for line in sys.stdin:
```

```
    fields = line.strip().split("\t")
    if len(fields) != 6:
        continue

    company_id, taxi_id = fields[0], fields[1]
    try:
        fleet_partial = int(fields[2])
        revenue_partial = float(fields[3])
        trips_partial = int(fields[4])
        distance_partial = float(fields[5])
    except ValueError:
        continue

    if current_taxi is None:
        current_company, current_taxi = company_id, taxi_id
    elif company_id != current_company or taxi_id != current_taxi:
        previous_company = current_company
        add_current_taxi()

        if company_id != previous_company:
            emit_current_company()
            company_fleet = 0
            company_revenue = 0.0
            company_trips = 0
            company_distance = 0.0

        current_company, current_taxi = company_id, taxi_id
        taxi_present = False
        taxi_revenue = 0.0
        taxi_trips = 0
        taxi_distance = 0.0

    taxi_present = taxi_present or fleet_partial > 0
    taxi_revenue += revenue_partial
    taxi_trips += trips_partial
    taxi_distance += distance_partial

add_current_taxi()
emit_current_company()
```

task3_job3_mapper.py

Task 3 | Task 3/task3_job3_mapper.py | 29 lines

```
#!/usr/bin/env python3
"""Job 3 mapper: replicate the small company table to three sort buckets.

Each company summary is emitted once for bucket 0, 1, and 2. Hadoop partitions
by bucket and sorts by revenue descending, so all three reducers receive the
same ordered company table. Each reducer later outputs only its rank slice.
"""

import sys


for line in sys.stdin:
    fields = line.strip().split("\t")
    if len(fields) != 6:
        continue

    company_id, revenue = fields[0], fields[1]
    try:
        int(company_id)
        float(revenue)
        int(fields[2])
        int(fields[3])
        float(fields[4])
        float(fields[5])
    except ValueError:
        continue

    for bucket in range(3):
        print("%d\t%s\t%s\t%s\t%s\t%s\t%s" % (bucket, revenue, company_id, fields[2], fields[3], fields[4], fields[5]))
```

task3_job3_reducer.py

Task 3 | Task 3/task3_job3_reducer.py | 50 lines

```
#!/usr/bin/env python3
"""Job 3 reducer: output one contiguous third of the global revenue ranking.

Input per reducer is the complete company table, already sorted by revenue
descending and company ID ascending for ties. Bucket 0 outputs the first rank
third, bucket 1 the middle third, and bucket 2 the final third. With bucket
digits partitioned across three reducers, concatenating part files in reducer
number order is therefore globally sorted.
"""

import sys


def emit_rank_slice(bucket, records):
    if bucket is None or not records:
        return

    record_count = len(records)
    start = bucket * record_count // 3
    end = (bucket + 1) * record_count // 3

    for fields in records[start:end]:
        # Remove the internal bucket field and restore the required six fields:
        # company, revenue, trips, fleet, revenue/taxi, average distance.
        print("%s\t%s\t%s\t%s\t%s\t%s" % (fields[2], fields[1], fields[3], fields[4], fields[5], fields[6]))


current_bucket = None
records = []

for line in sys.stdin:
    fields = line.strip().split("\t")
    if len(fields) != 7:
        continue

    try:
        bucket = int(fields[0])
    except ValueError:
        continue

    if current_bucket is None:
        current_bucket = bucket
    elif bucket != current_bucket:
        emit_rank_slice(current_bucket, records)
        current_bucket = bucket
        records = []

    records.append(fields)

emit_rank_slice(current_bucket, records)
```

task3-run.sh

Task 3 | Task 3/task3-run.sh | 74 lines

```
#!/bin/bash
# Task 3 - Company Performance Analysis: join, aggregate, then globally sort.

set -euo pipefail
cd "$(dirname "$0")"

STREAMING_JAR=/usr/lib/hadoop/hadoop-streaming.jar
TEMP_BASE=/tmp/bdp_assignment1_task3
JOB1_OUTPUT=$TEMP_BASE/job1_join
JOB2_OUTPUT=$TEMP_BASE/job2_aggregation
FINAL_OUTPUT=/Output/task3

chmod +x task3_job1_mapper.py task3_job1_reducer.py \
  task3_job2_mapper.py task3_job2_reducer.py \
  task3_job3_mapper.py task3_job3_reducer.py

# Remove stale intermediate/final directories so the script is end-to-end.
hadoop fs -rm -r -f "$TEMP_BASE"
hadoop fs -rm -r -f "$FINAL_OUTPUT"

# Job 1: reduce-side join on taxi_id. Partition by taxi_id; secondary-sort
# tag 0 metadata before tag 1 trip records so the reducer can stream the join.
hadoop jar "$STREAMING_JAR" \
  -D mapred.reduce.tasks=3 \
  -D stream.num.map.output.key.fields=2 \
  -D mapred.text.key.partitioner.options='-k1,1' \
  -D mapred.output.key.comparator.class=org.apache.hadoop.mapred.lib.KeyFieldBasedComparator \
  -D mapred.text.key.comparator.options='-k1,1n -k2,2n' \
  -file ./task3_job1_mapper.py \
  -mapper ./task3_job1_mapper.py \
  -file ./task3_job1_reducer.py \
  -reducer ./task3_job1_reducer.py \
  -input /Input/Taxis.txt \
  -input /Input/Trips.txt \
  -output "$JOB1_OUTPUT" \
  -partitioner org.apache.hadoop.mapred.lib.KeyFieldBasedPartitioner

# Job 2: aggregate by company while secondary-sorting by taxi_id. This lets
# reducers count distinct fleet members without holding a company-sized set.
hadoop jar "$STREAMING_JAR" \
  -D mapred.reduce.tasks=3 \
  -D stream.num.map.output.key.fields=2 \
  -D mapred.text.key.partitioner.options='-k1,1' \
  -D mapred.output.key.comparator.class=org.apache.hadoop.mapred.lib.KeyFieldBasedComparator \
  -D mapred.text.key.comparator.options='-k1,1n -k2,2n' \
  -file ./task3_job2_mapper.py \
  -mapper ./task3_job2_mapper.py \
  -file ./task3_job2_reducer.py \
  -reducer ./task3_job2_reducer.py \
  -input "$JOB1_OUTPUT" \
  -output "$JOB2_OUTPUT" \
  -partitioner org.apache.hadoop.mapred.lib.KeyFieldBasedPartitioner

# Job 3: replicate the small company-summary table to buckets 0,1,2. Each
# reducer receives a full revenue-descending order and emits its contiguous
```

```
# rank third. part-00000 + part-00001 + part-00002 is globally descending.
hadoop jar "$STREAMING_JAR" \
  -D mapred.reduce.tasks=3 \
  -D stream.num.map.output.key.fields=3 \
  -D mapred.text.key.partitioner.options='-k1,1' \
  -D mapred.output.key.comparator.class=org.apache.hadoop.mapred.lib.KeyFieldBasedComparator \
  -D mapred.text.key.comparator.options='-k1,1n -k2,2nr -k3,3n' \
  -file ./task3_job3_mapper.py \
  -mapper ./task3_job3_mapper.py \
  -file ./task3_job3_reducer.py \
  -reducer ./task3_job3_reducer.py \
  -input "$JOB2_OUTPUT" \
  -output "$FINAL_OUTPUT" \
  -partitioner org.apache.hadoop.mapred.lib.KeyFieldBasedPartitioner

# Only the assessed final directory remains.
hadoop fs -rm -r -f "$TEMP_BASE"

echo "Task 3 complete: $FINAL_OUTPUT"
```